



Rapport technique

Equipe: Diana, Ilan, Anthony

1. Introduction:

Dans le cadre de ce projet, on nous a donné comme mission de concevoir et développer le backend métier de la plateforme HealthAI Coach, une startup spécialisée dans le suivi de la santé et du bien-être basé sur la donnée.

L'objectif principal est de mettre en place une infrastructure technique permettant de collecter, nettoyer, stocker et exploiter des données hétérogènes issues de différentes sources (open, data, API, fichiers). Cette base de données servira à alimenter des outils d'analyse ainsi que de futurs modules d'intelligence artificielle.

Ce projet contribue dans une démarche industrielle avec des exigences fortes en termes de qualités des données, d'automatisation, de sécurité et de reproductibilité.

2. Contexte et analyse du besoin

HealthAI Coach évolue dans un marché en forte croissance où les utilisateurs recherchent des solutions personnalisées pour améliorer leur santé (nutrition, activité physique, sommeil).

Pour répondre à ces besoins, la plateforme doit être capable de manipuler plusieurs types de données:

- Profils utilisateurs (âge, poids, objectifs)
- Données nutritionnelles (aliments, macronutriments)
- Exercices physiques (types, intensité)
- Données de suivi (activité, progression, biométrie)

Le principal enjeu technique est de gérer des données hétérogènes provenant de différentes sources et formats (CSV, JSON), tout en garantissant leur qualité et leur cohérence.

Les objectifs sont donc :

- Mettre en place un pipeline ETL automatisé
- Concevoir une base de données relationnelle fiable
- Développer une API REST sécurisée
- Fournir un tableau de bord analytique

Choix technique et architecture:

3.1 Architecture globale

L'architecture mise en place repose sur un pipeline de traitement des données structuré en plusieurs étapes :

1. Collecte des données depuis différentes sources
2. Transformation et nettoyage via un processus ETL
3. Stockage dans une base de données relationnelle (PostgreSQL)
4. Exposition via une API REST (Laravel)
5. Visualisation via un tableau de bord (Grafana)

Cette architecture garantit une separation claire des responsabilités et facilite la maintenance ainsi que l'évolution du système.

3.2 Choix des technologies

Plusieurs technologies ont été sélectionnées pour répondre aux besoins du projet :

- Laravel : développement d'une API REST rapide et documentée
- Python : utilise pour le traitement des données et le développement du pipeline ETL
- PostgreSQL : base de données relationnelle robuste et performante
- Pandas : manipulation et nettoyage des données
- Docker : conteneurisation de l'environnement pour assurer la reproductibilité
- Grafana : visualisation des données

Ces choix ont été guidés par leur popularité, leur robustesse et leur adéquation avec les problématiques de data engineering.



4. Gestion des données

4.1 Source de données

Les données utilisées proviennent de plusieurs sources :

- Datasets Kaggle (nutrition, fitness)
- API d'exercices sportifs

Food: <https://www.kaggle.com/datasets/adilshamim8/daily-food-and-nutrition-dataset>

Exercice: <https://www.kaggle.com/datasets/exercisedb/fitness-exercises-dataset?resource=download>

Ces données sont disponibles sous différents formats (CSV, JSON), ce qui nécessite un traitement adapté.

4.2 Pipeline ETL

La pipeline ETL (Extract, Transform, Load) a été conçu pour automatiser le traitement des données.

- Extraction : récupération des fichiers et des données depuis Google Drive
- Transformation :
 - Suppression des doublons
 - Gestion des valeurs manquantes
 - Renommage des colonnes au mauvais nom
- Chargement : insertion des données nettoyés dans PostgreSQL

Ce pipeline peut être exécuté automatiquement via des scripts planifiés.

4.3 Qualité des données

Pour garantir la fiabilité des données, plusieurs mécanismes ont été mis en place :

- Validation des schémas
- Contrôle des types de données
- Logs d'erreurs

Ces étapes sont essentielles pour assurer la pertinence des analyses futures.

5. Modélisation des données

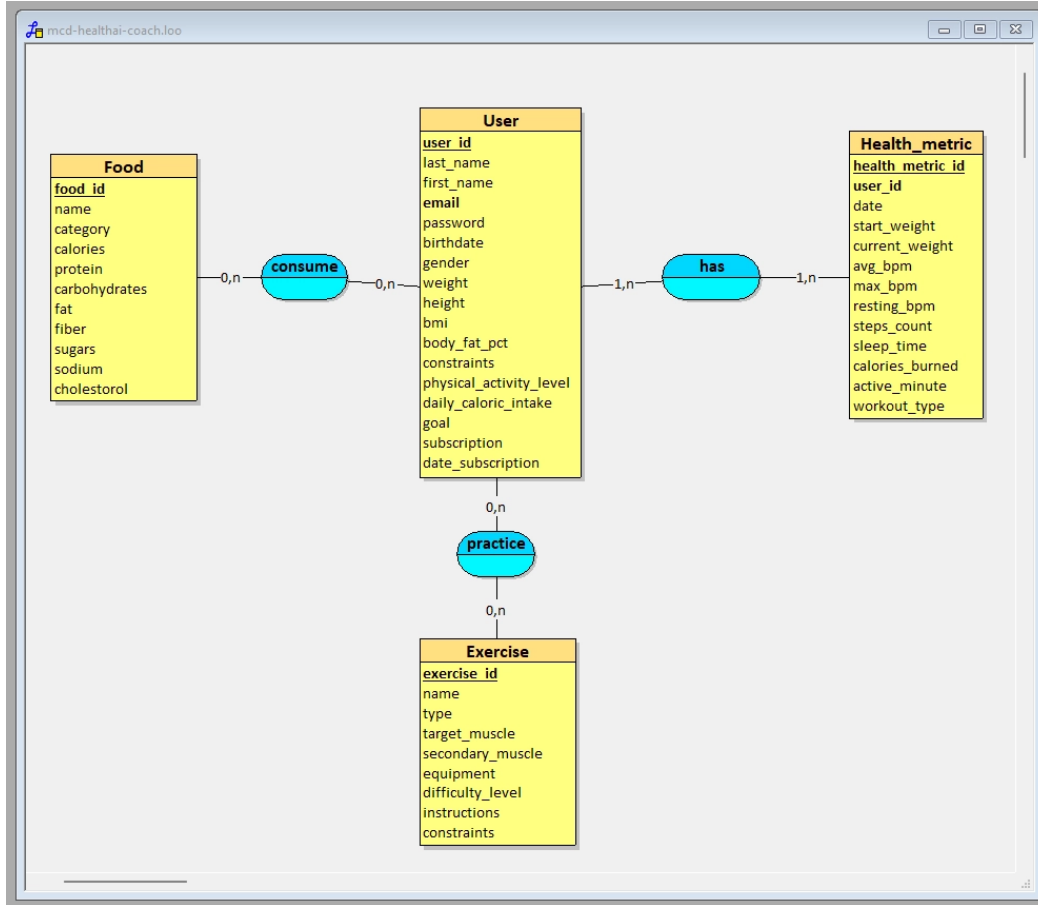
La base de données a été conçue selon une approche relationnelle.

Les principales entités sont :

- Users : informations des utilisateurs
- Foods : données nutritionnelles
- Exercices : catalogue d'exercices
- Metrics : données de suivi

Les relations entre ces tables permettent de croiser les données pour produire des analyses pertinentes.

Le modèle a été normalisé afin d'éviter la redondance et d'optimiser les performances.



6. API et exploitation des données

Une API REST a été développée pour permettre l'accès aux données.

Les principales fonctionnalités incluent :

- CRUD utilisateurs
- Consultation des données nutritionnelles
- Accès aux exercices
- Récupération des métriques

L'API est sécurisée via un système d'authentification (JWT) et documentée grâce à OpenAPI (Swagger), facilitant son utilisation par les développeurs front-end et partenaire.

7. Visualisation et dashboard



Avec Grafana, un tableau de bord interactif a été mis en place pour visualiser les données.

Les indicateurs affichés incluent :

- Répartition des utilisateurs
- Progression des performances
- Tendances nutritionnelles
- KPIs business

L'interface a été conçue pour être accessible et compréhensible par des utilisateurs non techniques.

8. Difficultés rencontrées :

Plusieurs difficultés ont été rencontrées lors du projet :

- Diana : Création des requêtes avec PostgreSQL
- Ilan : Gestion des permissions, Déploiement Railway, Script Bat Automatisation local
- Anthony : Gestion des connexions à la base de données, recherche sur railway,

9. Perspective d'évolution :

Plusieurs axes d'amélioration sont envisagés :

- Intégration de modules d'intelligence artificielle
- Passage à une architecture microservices



- Optimisation des performances
- Amélioration de la sécurité

Ces évolutions permettront de renforcer la valeur de la plateforme.

10. Conclusion

Ce projet a permis de concevoir une solution complète de gestion de données, allant de la collecte des visualisations, et au déploiement.

L'infrastructure mise en place répond aux exigences du cahier des charges et constitue une base solide pour le développement futur de fonctionnalités avancées, notamment en intelligence artificielle.